

SENAI

Serviço Nacional de Aprendizagem Indústria
Escola Senai Luiz Varga - Limeira

DOCUMENTAÇÃO TÉCNICA

Sistema de Gestão para Confecção

confecção

Kayque Costa da Silva

Programação Back End

2026

1. Visão Geral do Projeto

O confecção é um sistema de gestão empresarial (ERP) desenvolvido para empresas do setor de confecção. A aplicação centraliza o controle de clientes, fornecedores, produtos, insumos, estoques e pedidos em um painel administrativo web completo e responsivo.

1.1 Objetivo

Automatizar e digitalizar os processos operacionais de uma confecção — desde o cadastro de fornecedores e controle de insumos até a emissão de pedidos com cálculo automático de totais — eliminando planilhas manuais e centralizando a informação em um único sistema.

1.2 Stack Tecnológico

Camada	Tecnologia	Versão
Backend Framework	Laravel	^13.0
Admin Panel	Filament	^5.0
Linguagem	PHP	^8.3
Banco de Dados	SQLite (dev) / MySQL (prod)	—
Autenticação / ACL	Spatie Laravel Permission	^7.2
Build Front-end	Vite + NPM	—
Envio de E-mail	Laravel Mail (Markdown)	—
Filas	Laravel Queue (database driver)	—

2. Estrutura de Diretórios

O projeto segue a estrutura padrão Laravel com organização por domínio dentro de app/Filament/Resources:

```

confeccao2/
├── app/
│   ├── Filament/
│   │   ├── Resources/
│   │   │   ├── Clientes/
│   │   │   ├── Estoques/
│   │   │   ├── Fornecedores/
│   │   │   ├── Insumos/
│   │   │   ├── Pedidos/
│   │   │   ├── Permissions/
│   │   │   ├── Produtos/
│   │   │   ├── Roles/
│   │   │   └── Users/
│   │   └── Widgets/
│   │       ├── StatsOverview.php
│   │       ├── ClienteChart.php
│   │       ├── FornecedorChart.php
│   │       ├── InsumoChart.php
│   │       └── PedidoChart.php
│   ├── Mail/
│   │   └── PedidoRecebidoMail.php
│   └── Models/
│       ├── Cliente.php
│       ├── Estoques.php
│       ├── Fornecedor.php
│       ├── Insumo.php
│       ├── ItemPedido.php
│       ├── Pedido.php
│       ├── Produto.php
│       └── User.php
├── database/
│   └── migrations/
├── resources/
│   └── views/
│       └── emails/

```

Cada Resource do Filament é organizado em quatro sub-pastas:

Pasta	Responsabilidade
Pages/	Define as rotas (List, Create, Edit, View)
Schemas/	Formulários e Infolists de exibição
Tables/	Configuração das listagens (colunas, filtros)
*Resource.php	Ponto central: navegação, permissões, relações

3. Modelo de Dados

O sistema utiliza 9 tabelas principais. A seguir estão os schemas de cada tabela com seus campos e tipos.

3.1 Tabela: clientes

Campo	Tipo	Restrição	Descrição
id	BIGINT	PK, auto	Identificador único
nome	VARCHAR	NOT NULL	Nome completo do cliente
email	VARCHAR	UNIQUE	E-mail de contato
telefone	VARCHAR	nullable	Telefone de contato
documento	VARCHAR	nullable	CPF ou CNPJ
tipo	ENUM	fisica/juridica	Tipo de pessoa
created_at / updated_at	TIMESTAMP	—	Controle de auditoria

3.2 Tabela: fornecedores

Campo	Tipo	Restrição	Descrição
id	BIGINT	PK, auto	Identificador único
nome	VARCHAR	NOT NULL	Razão social
nome_fantasia	VARCHAR	nullable	Nome fantasia
email	VARCHAR	UNIQUE	E-mail de contato
telefone	VARCHAR	nullable	Telefone
documento	VARCHAR	nullable	CNPJ / CPF

3.3 Tabela: produtos

Campo	Tipo	Restrição	Descrição
id	BIGINT	PK, auto	Identificador único
nome	VARCHAR	NOT NULL	Nome do produto
referencia	VARCHAR	UNIQUE	Código (ex: CAM-PRE-001)
preco	DECIMAL(10,2)	nullable	Preço de venda
estoque	INTEGER	default 0	Saldo atual em estoque

3.4 Tabela: insumos

Campo	Tipo	Restrição	Descrição
id	BIGINT	PK, auto	Identificador único
nome	VARCHAR	NOT NULL	Nome do insumo

Campo	Tipo	Restrição	Descrição
unidade_medida	VARCHAR	NOT NULL	kg, m, L, un, etc.
preco	DECIMAL(10,2)	nullable	Custo unitário
estoque	DECIMAL(10,2)	default 0	Quantidade em estoque

3.5 Tabela: pedidos

Campo	Tipo	Restrição	Descrição
id	BIGINT	PK, auto	Identificador único
cliente_id	BIGINT	FK → clientes	Cliente do pedido
status	VARCHAR	default 'pendente'	pendente / em_producao / finalizado
total	DECIMAL(10,2)	nullable	Valor total calculado

3.6 Tabela: item_pedidos

Campo	Tipo	Restrição	Descrição
id	BIGINT	PK, auto	Identificador único
pedido_id	BIGINT	FK → pedidos	Pedido pai (cascade delete)
produto_id	BIGINT	FK → produtos	Produto do item
quantidade	INTEGER	NOT NULL	Quantidade solicitada
preco_unitario	DECIMAL(10,2)	NOT NULL	Preço no momento da venda

3.7 Tabela: estoques

Campo	Tipo	Restrição	Descrição
id	BIGINT	PK, auto	Identificador único
produto_id	BIGINT	FK → produtos	Produto relacionado
tipo	ENUM	entrada / saida	Tipo do movimento
quantidade	INTEGER	NOT NULL	Quantidade movimentada
observacao	VARCHAR	nullable	Observação livre

3.8 Relacionamentos (Eloquent)

Model	Tipo	Relacionamento
Pedido	belongsTo	→ Cliente
Pedido	hasMany	→ ItemPedido
ItemPedido	belongsTo	→ Pedido
ItemPedido	belongsTo	→ Produto
Estoques	belongsTo	→ Produto

4. Módulos do Sistema

4.1 Navegação e Grupos

O menu lateral do Filament está organizado em três grupos temáticos:

Grupo	Resources	Acesso (Role)
Cadastros Gerais	Clientes, Fornecedores	RH
Controle de Estoque	Insumos, Produtos, Estoques	Gerente
Vendas	Pedidos	Todos autenticados
Administração	Users, Roles, Permissions	Super-admin

4.2 Controle de Acesso (RBAC)

O sistema usa o pacote Spatie Laravel Permission para controle de acesso baseado em papéis (RBAC). Cada Resource implementa o método `canAccess()` que verifica o papel do usuário autenticado:

```
// ClienteResource.php - acesso somente para papel 'RH'
public static function canAccess(): bool
{
    return auth()->user()->hasRole('RH') ?? false;
}

// InsumoResource.php / EstoquesResource.php - papel 'Gerente'
public static function canAccess(): bool
{
    return auth()->user()->hasRole('Gerente') ?? false;
}
```

4.3 Módulo de Pedidos

O módulo de pedidos é o mais complexo do sistema. Destaca-se o cálculo automático de total via Filament Repeater:

- O formulário de pedido contém um Repeater com itens (produto, quantidade, preço unitário).
- Quando quantidade ou preço unitário são alterados (`onBlur`), o método `calcularTotal()` é disparado.
- O método percorre todos os itens do Repeater e atualiza o campo `'total'` em tempo real.
- O status do pedido aceita três valores: Pendente, Em Produção e Finalizado.
- Na listagem, cada status exibe um badge colorido (`warning`, `info`, `success`).

```
public static function calcularTotal(Get $get, Set $set): void
{
    $itens = $get('itens') ?? [];
    $total = 0;
    foreach ($itens as $item) {
        $quantidade = (float) ($item['quantidade'] ?? 0);
        $preco = (float) ($item['preco_unitario'] ?? 0);
        $total += $quantidade * $preco;
    }
    $set('total', number_format($total, 2, ',', '.'));
}
```

}

4.4 Módulo de Insumos

O cadastro de insumos suporta uma ampla variedade de unidades de medida, organizadas em grupos no campo Select:

- Massa/Peso: Quilograma (kg), Grama (g), Miligrama (mg), Tonelada (t)
- Volume/Líquidos: Litro (L), Mililitro (mL), Metro cúbico (m³)
- Comprimento/Área: Metro (m), Centímetro (cm), Metro quadrado (m²)
- Unidades diversas: Unidade (un), Par, Peça, Rolo, Bobina, entre outros

5. Dashboard e Widgets

O painel principal exibe cinco widgets configurados para atualização automática. O intervalo de polling é de 10 segundos para as métricas de resumo.

5.1 StatsOverview — Métricas Gerais

Métrica	Ícone	Cor
Total de Clientes	heroicon-o-users	primary
Total de Fornecedores	heroicon-o-truck	success
Total de Insumos	heroicon-o-shopping-cart	warning
Total de Pedidos	heroicon-o-document-text	danger
Total de Produtos	heroicon-o-cube	info
Total de Estoques	heroicon-o-archive-box	secondary
Total de Itens de Pedido	heroicon-o-clipboard-document-list	primary

5.2 Gráficos (ChartWidget)

Além do StatsOverview, existem quatro gráficos de linha exibindo a evolução mensal dos últimos 6 meses:

- PedidoChart — Pedidos por status (Pendente, Em Produção, Finalizado) com cores distintas por série.
- ClienteChart — Evolução de novos clientes cadastrados mês a mês.
- FornecedorChart — Evolução de novos fornecedores cadastrados mês a mês.
- InsumoChart — Evolução de novos insumos cadastrados mês a mês.

Todos os gráficos utilizam Carbon para geração dinâmica das labels de data no formato 'Mês/Ano' com tradução para pt-BR.

6. Sistema de E-mail

O sistema implementa notificação por e-mail ao cliente quando um pedido é criado ou atualizado. A classe PedidoRecebidoMail utiliza o sistema de Mailables do Laravel com template Markdown.

6.1 Classe PedidoRecebidoMail

```
class PedidoRecebidoMail extends Mailable
{
    use Queueable, SerializesModels;
```

```
public function __construct(
    public Pedido $pedido
) {}

public function envelope(): Envelope
{
    return new Envelope(
        subject: "Atualização do seu Pedido #{$this->pedido->id} 🧵",
    );
}

public function content(): Content
{
    return new Content(
        markdown: 'emails.pedido-recebido',
    );
}
}
```

O envio é enfileirado via Laravel Queue (driver: database), garantindo que a operação principal não seja bloqueada pela latência do servidor SMTP.

7. Configuração e Instalação

7.1 Pré-requisitos

- PHP >= 8.3 com extensões: pdo, mbstring, xml, bcmath, curl
- Composer >= 2.x
- Node.js >= 18.x e NPM
- SQLite (desenvolvimento) ou MySQL/PostgreSQL (produção)

7.2 Instalação

O projeto inclui um script composer setup que automatiza todo o processo de instalação:

```
# Clonar o repositório
git clone <url-do-repositorio> confeccao2
cd confeccao2

# Executar setup completo (instala dependências,
# gera .env, chave da aplicação e roda migrations)
composer setup

# Criar usuário administrador (interativo)
php artisan make:filament-user
```

7.3 Execução em Desenvolvimento

O script composer dev inicia simultaneamente o servidor PHP, fila de jobs, monitoramento de logs e servidor Vite:

```
composer dev

# Equivalente a:
# php artisan serve
# php artisan queue:listen --tries=1 --timeout=0
# php artisan pail --timeout=0
# npm run dev
```

7.4 Variáveis de Ambiente (.env)

Variável	Padrão (dev)	Descrição
APP_ENV	local	Ambiente da aplicação
APP_DEBUG	true	Exibir erros detalhados
DB_CONNECTION	sqlite	Driver do banco de dados
QUEUE_CONNECTION	database	Driver da fila de jobs
MAIL_MAILER	log	Driver SMTP (log = sem envio real)
SESSION_DRIVER	database	Driver de sessão
CACHE_STORE	database	Driver de cache

8. Observações Técnicas

8.1 Typographical Note — Nome da Tabela

A tabela de itens de pedido foi criada com o nome `item_pedidos` (tipo de 'pedidos'). O model correspondente é `ItemPeido.php`. Isso não afeta o funcionamento, mas deve ser corrigido em uma futura migration caso a base de dados seja refatorada.

8.2 Filament v5

O projeto usa Filament na versão ^5, que introduz mudanças significativas na API em relação à v3. Destacam-se:

- Uso de `Filament\Schemas\Schema` no lugar de `Forms\Form` e `Infolists\Infolist`.
- `Resources` com namespace separado por domínio (ex: `Resources/Pedidos/PedidoResource.php`).
- `Schemas` centralizados em sub-pastas `Schemas/` para reaproveitamento entre `Form` e `Infolist`.
- Heroicons referenciados via enum: `Heroicon::OutlinedRectangleStack`.

8.3 Polling do Dashboard

O widget `StatsOverview` define `pollingInterval = '10s'`, garantindo que os totais no dashboard sejam atualizados automaticamente a cada 10 segundos sem necessidade de refresh manual pela equipe operacional.

9. Dependências do Projeto

9.1 Produção (composer.json → require)

Pacote	Versão	Finalidade
laravel/framework	^13.0	Framework principal
filament/filament	^5.0	Painel administrativo
filament/widgets	^5.0	Gráficos e widgets do dashboard
spatie/laravel-permission	^7.2	RBAC — roles e permissions
laravel/tinker	^3.0	REPL para debug e seeds

9.2 Desenvolvimento (require-dev)

Pacote	Versão	Finalidade
phpunit/phpunit	^12.5	Framework de testes
fakerphp/faker	^1.23	Geração de dados falsos
laravel/pint	^1.27	Code style (PSR-12)

Pacote	Versão	Finalidade
laravel/pail	^1.2	Log viewer em tempo real
nunomaduro/collision	^8.6	Output amigável de erros
mockery/mockery	^1.6	Mocks para testes